

Understanding Namespaces In Docker



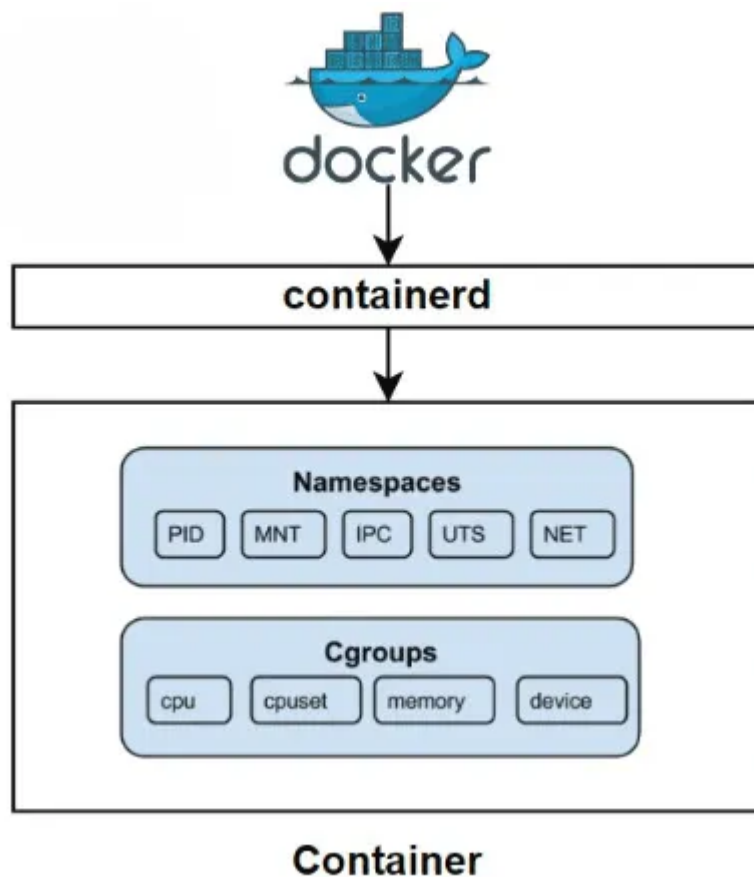
@Harsh · [Follow](#)

6 min read · Jan 9, 2024

 Listen

 Share

... More



INTRODUCTION

In the realm of containerization, Docker's use of namespaces is a powerful and foundational concept that underlies the platform's ability to isolate and share resources efficiently. To truly grasp the significance of Docker's namespaces, let's embark on a journey comparing it to the general process of sharing resources among traditional processes.

Understanding General Process Sharing Resources

In a general computing environment, processes share the same set of resources, creating potential conflicts and challenges in resource management.

Imagine multiple processes running on a host system without any isolation. These processes may inadvertently interfere with each other, leading to issues like resource contention, security vulnerabilities, and difficulties in managing dependencies.

In this scenario, processes share a common namespace, leading to a lack of isolation and independence. Resource conflicts, such as two processes trying to access the same memory address or conflicting file access, can result in unstable and unpredictable behavior.

LINUX NAMESPACES

Namespaces are one of a feature in the Linux Kernel and a fundamental aspect of containers on Linux. On the other hand, namespaces provide a layer of isolation. **Namespaces** are a feature of the Linux kernel that partitions kernel resources such that one set of processes sees one set of resources while another set of processes sees a different set of resources.

This is a feature of Linux that allows us to create something like a virtual machine or a tool like docker, quite similar to the function of virtual machine tools. Solomon Hyke, the founder of Docker, utilizes these powerful features of linux kernel and created the revolutionary tool that brings the new layer of agility and isolation in tech world.

DOCKER

Docker uses namespaces to provide the isolated workspace called the container. When you run a container, Docker creates a set of namespaces for that container.

This means that each container has its own isolated namespace for processes, network, mount points, and more.

Let's explore the key namespaces in Docker and how they enhance resource isolation:

1. PID Namespace: Process isolation (PID: Process ID)
 - General Process Sharing: All processes on a system share the same PID namespace.

- Docker's Approach: Each container has its own PID namespace, allowing processes within the container to have their unique set of process IDs.

2. Net Namespace: Managing network interfaces (NET: Networking)

- General Process Sharing: Processes often share the same network namespace, leading to potential conflicts.
- Docker's Approach: Each container gets its isolated network namespace, enabling independent network configurations and avoiding interference between containers

3. IPC Namespace: Managing access to IPC resources (IPC: InterProcess Communication)

- General Process Sharing: Processes share the same IPC namespace, leading to shared resources and potential conflicts.
- Docker's Approach: Containers have separate IPC namespaces, providing isolation for inter-process communication.

4. MNT Namespace: Managing filesystem mount points (MNT: Mount)

- General Process Sharing: Processes typically share the same mount namespace, making file system management challenging.
- Docker's Approach: Containers have their own mount namespace, ensuring that file systems are isolated and can be managed independently.

5. UTS Namespace: Different host and domain names (UTS: Unix Timesharing System)

- General Process Sharing: Processes usually share the same UTS namespace, making hostname conflicts possible.
- Docker's Approach: Containers have their own UTS namespace, allowing each container to have a unique hostname.

6. USER Namespace:

- **General Process Sharing:** Processes can see the userIDs and groupIDs of other processes in same namespaces.
- **Docker's Approach:** Isolate the user IDs and group IDs of processes in a container. This means that processes in one container cannot see the user IDs and group IDs of processes in another container.

By leveraging namespaces, Docker achieves enhanced resource isolation, making containers lightweight, portable, and secure.

Practical For Better Understanding

The containers is considered as the separate OS because it has its own Namespaces and resources. Although it share the resources from base OS with the help of c-groups but having isolated and own namespaces makes it a OS.

The `docker run` command initializes the new process and create the separate namespace for that process so it can use the resources in isolation.

```
[root@ip-172-31-32-150 ~]# hostname
ip-172-31-32-150.ap-south-1.compute.internal
[root@ip-172-31-32-150 ~]#
[root@ip-172-31-32-150 ~]# docker run -it centos:7
[root@6a646179b179 /]# hostname
6a646179b179
[root@6a646179b179 /]#
```

As we can see that the hostname inside the container is different from that of base hostname because of the `uts` namespace.

```
[root@2a3b02d6e920 /]# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 172.17.0.2  netmask 255.255.0.0  broadcast 172.17.255.255
    ether 02:42:ac:11:00:02  txqueuelen 0  (Ethernet)
    RX packets 1253  bytes 33176867 (31.6 MiB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 767  bytes 55219 (53.9 KiB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

The IP of the container is different from that of host IP because of `net` namespace.

As soon as we start the container it initializes the process of whatever we have mention in CMD of the image.

```
[root@ip-172-31-32-150 ~]# ps -aux | grep bash
ec2-user  33916  0.0  0.5 233060  5016 pts/0    Ss   04:48   0:00 -bash
root      33990  0.0  0.5 233188  4992 pts/0    S    04:49   0:00 -bash
root      34013  0.0  0.5 233212  5064 pts/0    S    04:49   0:00 -bash
root      34612  0.0  0.2  11844  2832 pts/0    Ss+  05:02   0:00 /bin/bash
root      34753  0.0  0.2 222312  2024 pts/0    S+   05:07   0:00 grep --color=auto bash
[root@ip-172-31-32-150 ~]#
```

We can see that 34612 pid were in actual the pid of the container. As soon as we kill that process we can see our container automatically removed.

```
[root@ip-172-31-32-150 ~]#
[root@ip-172-31-32-150 ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS   NAMES
6a646179b179   centos:7  "/bin/bash"             7 minutes ago Up 7 minutes   crazy_mclean
[root@ip-172-31-32-150 ~]#
[root@ip-172-31-32-150 ~]# kill -9 34612
[root@ip-172-31-32-150 ~]#
[root@ip-172-31-32-150 ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS   NAMES
[root@ip-172-31-32-150 ~]#
```

There is a command in the linux where you can see all the namespaces associated with the particular process.

- `lsns`

```
[root@ip-172-31-32-150 ~]# lsns
NS TYPE  NPROCS  PID USER          COMMAND
026531834 time    108     1 root          /usr/lib/systemd/systemd --switched-root --system --deserialize=32
026531835 cgroup   108     1 root          /usr/lib/systemd/systemd --switched-root --system --deserialize=32
026531836 pid      108     1 root          /usr/lib/systemd/systemd --switched-root --system --deserialize=32
026531837 user     108     1 root          /usr/lib/systemd/systemd --switched-root --system --deserialize=32
026531838 uts      101     1 root          /usr/lib/systemd/systemd --switched-root --system --deserialize=32
026531839 ipc      108     1 root          /usr/lib/systemd/systemd --switched-root --system --deserialize=32
026531840 net      108     1 root          /usr/lib/systemd/systemd --switched-root --system --deserialize=32
026531841 mnt       95     1 root          /usr/lib/systemd/systemd --switched-root --system --deserialize=32
026531862 mnt        1     20 root          kdevtmpfs
026532200 mnt        1    1595 root          /usr/lib/systemd/systemd-udevd
026532201 uts        1    1595 root          /usr/lib/systemd/systemd-udevd
026532209 mnt        1    1934 root          /sbin/auditd
026532210 mnt        1    1922 systemd-resolve /usr/lib/systemd/systemd-resolved
026532220 mnt        2    1953 dbus          /usr/bin/dbus-broker-launch --scope system --audit
026532221 mnt        1    2236 chrony        /usr/sbin/chronyd -F 2
```

These all are the namespaces that is been associated with these processes.

On running `docker run` command on the terminal, we can get the new bash which can be proved by seeing in pids that a new `bash` process has been initiated.

```

[root@ip-172-31-32-150 ~]# docker run -it centos:7
[root@9cc8fd926937 /]#
[root@9cc8fd926937 /]# [root@ip-172-31-32-150 ~]#
[root@ip-172-31-32-150 ~]# ps -aux | grep bash
ec2-user   33916  0.0  0.5 233060  5016 pts/0    ss   04:48   0:00 -bash
root       33990  0.0  0.5 233188  4992 pts/0    s    04:49   0:00 -bash
root       34013  0.0  0.5 233212  5068 pts/0    s    04:49   0:00 -bash
root       35424  0.0  0.5 233100  5088 pts/0    s    05:18   0:00 bash
root       36174  0.2  0.2  11844  2916 pts/0    ss+  05:29   0:00 /bin/bash
root       36206  0.0  0.2 222312  2024 pts/0    s+   05:29   0:00 grep --color=auto bash
[root@ip-172-31-32-150 ~]#
[root@ip-172-31-32-150 ~]#

```

We can now give separate namespaces to this process of ID 36174 using the command `nsenter`.

Now without using any of the docker command we can go inside the process using `nsenter` command as we know that this process already has separate namespaces.

```

root       36174  0.2  0.2  11844  2916 pts/0    ss+  05:29   0:00 /bin/bash
root       36206  0.0  0.2 222312  2024 pts/0    s+   05:29   0:00 grep --color=auto bash
[root@ip-172-31-32-150 ~]#
[root@ip-172-31-32-150 ~]#
[root@ip-172-31-32-150 ~]# nsenter -t 36174 -n
[root@ip-172-31-32-150 ~]# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 172.17.0.2  netmask 255.255.0.0  broadcast 172.17.255.255
    ether 02:42:ac:11:00:02  txqueuelen 0  (Ethernet)
    RX packets 12  bytes 1016 (1016.0 B)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 0  bytes 0 (0.0 B)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

```

Now we can see that we get the new IP of the container without using any docker command. `nsenter -t <target pid> <option for target ns>`, this command takes you inside the process and inside the namespace of that process.

This is why we get the new IP of the container even without using any docker command. That means docker behind the scene using this command for getting the IP.

```

[root@ip-172-31-32-150 ~]# nsenter -t 36174 -u
[root@9cc8fd926937 ~]# hostname
9cc8fd926937
[root@9cc8fd926937 ~]#

```

Similarly we get the different hostname for this container without going inside the container.

Now with the command `nsenter -t <target ip> -a`, we can get access to all the namespace of that process. It is same to the command `docker attach <name>`

```
[root@ip-172-31-32-150 ~]# nsenter -t 36174 -a
[root@9cc8fd926937 /]#
[root@9cc8fd926937 /]#
```

Hence it has been proved.

Cgroups

We could have created a process separate from the other process with Linux namespaces. But if we create multiple namespaces, then how can we limit the resources of each namespace so that it doesn't take up the resources of another namespace?

Cgroups (abbreviated from **control groups**) is a Linux kernel feature that limits, accounts for, and isolates the resource usage (CPU, memory, disk I/O, network, etc.) of a collection of processes.

Conclusion

In conclusion, Docker's namespace concept transforms the landscape of resource isolation, offering a more efficient and secure approach to containerization. As we continue to explore container orchestration and deployment strategies, understanding namespaces remains a key pillar in unlocking the full potential of Docker. 🚀🐳

#Docker #Containerization #NamespaceConcept #ResourceIsolation